

## In-Class Hands-On: Binary Search Tree (BST) Exercise **with Solutions**

This is a **group assignment** that you work on in groups of 2-3 persons. Solve each exercise and discuss the solution approach used as a group.

Write the solutions on a **piece of paper**. Please **close all computers and phones and put them aside**. Instead, please **call the teaching assistants (TAs) over if you need help**. The goal is to learn (its not about all necessarily being correct; any reasonably thoughtful submission gets a full point).

**Rewrite up your final answers clearly on a single sheet of paper** (use both sides if needed; if you need to, you can get a second piece of paper). **Write the name of all group members on the top of the sheet(s).**

**Hand in your answer sheet(s) to the TA.**

---

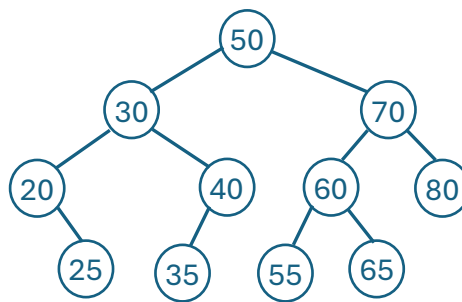
You start with an empty *Binary Search Tree (BST)*.

### 1. Insert

Insert the following keys *in this order* into the binary tree:

50, 30, 20, 25, 70, 60, 40, 35, 65, 80, 55

- Draw the resulting BST.
- In your own words, describe the approaches used for inserting (be as concise as possible).



Start at the root. If the tree is empty, insert the value as the root. Otherwise, compare the value with the current node's value: go left if smaller, right otherwise. Repeat until a missing child is found, and insert the new node with the given value there.

### 2. Find/Search

Search for each of the following keys in the tree:

- a) 65
- b) 40
- c) 25

- For each key, show the *path followed in the tree* and state whether the key was found.

a) 50 – 70 – 60 – 65 : found  
b) 50 – 30 – 40 : found  
c) 50 – 30 – 20 – 25 : found

- In your own words, describe the approaches used for searching (be as concise as possible).

Start at the root. If the tree is empty, the value is not found. Otherwise, compare the value with the current node's value: if equal, then found; if smaller, go to the left child; otherwise, go to the right child. Repeat until the value is found or no more node is reached (not found).

### 3. Find before key (in-order predecessor)

Find the *previous key in sorted order* (in-order predecessor) for:

- a) 70
- b) 55
- c) 20

- For each key, state the predecessor if it exists, and show the *path followed in the tree*.

a) 70 – 60 – **65** : found (65)  
b) 55 – 60 – 70 – **50** : found (50)  
c) 20 – 30 – 50 : not found

- In your own words, describe the approaches used for finding the predecessor (be as concise as possible). Were there any special cases you needed to consider?

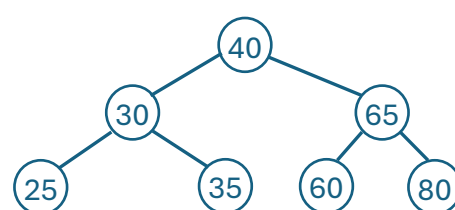
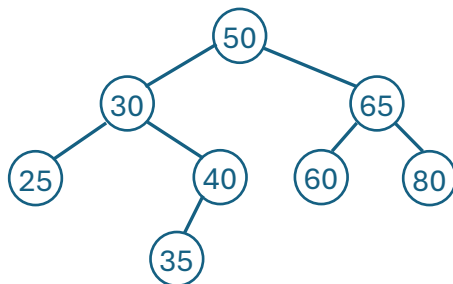
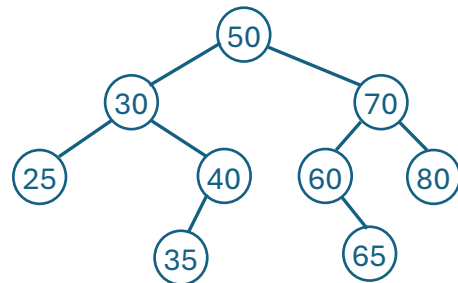
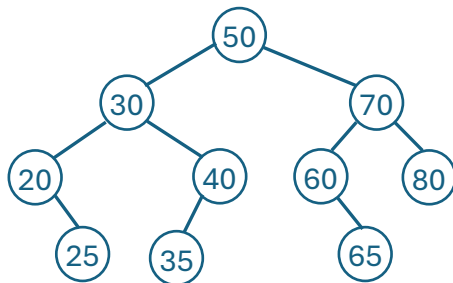
Start at the given node. If it has a left child, go to the left once, then keep going right until there is no right child—the final node is the predecessor. If there is no left child, move up the tree until you come from a right child; that parent is the predecessor. If no such node exists, there is no predecessor.

### 4. Delete

In the tree, perform the following deletions one after another (on the modified tree).

- a) 55
- b) 20
- c) 70
- d) 50

- Draw the tree *after each deletion*.



- In your own words, describe the approaches used for deleting (be as concise as possible). What were the special cases you needed to consider, and how did you handle them differently?

Start at the root and locate the node to delete. If not found, stop. Otherwise:

- If the node has no children, remove it.
- If it has one child, replace it with its child.
- If it has two children, find its predecessor, copy its value into the node, then delete the predecessor node (which will have at most one child) using the same rules.